

**Name-Gopal Kumar Jha**

**Sec-6**

**Qid-23030896**

**16. Add**

**Two Numbers**

You are given two non-empty linked lists representing two non-negative integers. Add the numbers and return the sum as a linked list.

```
#include <iostream>
```

```
using namespace std;
```

```
// Definition for singly-linked list
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode* next;
```

```
    ListNode(int x) {
```

```
        val = x;
```

```
        next = NULL;
```

```
    }
```

```
};
```

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
```

```
    ListNode dummy(0);
```

```
    ListNode* current = &dummy;
```

```
    int carry = 0;
```

```

while (l1 != NULL || l2 != NULL || carry) {

    int sum = carry;

    if (l1 != NULL) {

        sum += l1->val;

        l1 = l1->next;

    }

    if (l2 != NULL) {

        sum += l2->val;

        l2 = l2->next;

    }

    carry = sum / 10;

    current->next = new ListNode(sum % 10);

    current = current->next;

}

return dummy.next;

}

```

```

// Function to print linked list
void printList(ListNode* head) {

    while (head != NULL) {

        cout << head->val << " ";

    }
}

```

```

        head = head->next;
    }
}

int main() {
    // Number 342 -> 2 -> 4 -> 3
    ListNode* l1 = new ListNode(2);
    l1->next = new ListNode(4);
    l1->next->next = new ListNode(3);

    // Number 465 -> 5 -> 6 -> 4
    ListNode* l2 = new ListNode(5);
    l2->next = new ListNode(6);
    l2->next->next = new ListNode(4);

    ListNode* result = addTwoNumbers(l1, l2);

    printList(result);

    return 0;
}

```

## 17. Longest Substring Without Repeating Characters

Given a string *s*, find the length of the longest substring without repeating characters.

```

#include <iostream>

#include <unordered_set>

using namespace std;

```

```
int lengthOfLongestSubstring(string s) {  
    unordered_set<char> chars;  
  
    int left = 0;  
    int maxLength = 0;  
  
    for (int right = 0; right < s.length(); right++) {  
  
        // Remove characters until duplicate is removed  
        while (chars.count(s[right])) {  
            chars.erase(s[left]);  
            left++;  
        }  
  
        chars.insert(s[right]);  
  
        maxLength = max(maxLength, right - left + 1);  
    }  
  
    return maxLength;  
}  
  
int main() {  
    string s = "abcabcbb";  
  
    cout << lengthOfLongestSubstring(s);
```

```
    return 0;
}
```

## 18. Longest Palindromic Substring

Given a string *s*, return the longest substring that is a palindrome.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
string expandFromCenter(string s, int left, int right) {
    while (left >= 0 && right < s.size() && s[left] == s[right]) {
        left--;
        right++;
    }

    return s.substr(left + 1, right - left - 1);
}
```

```
string longestPalindrome(string s) {
    if (s.empty())
        return "";

    string longest = "";

    for (int i = 0; i < s.size(); i++) {

        // Odd length palindrome
        string odd = expandFromCenter(s, i, i);
```

```

        // Even length palindrome
        string even = expandFromCenter(s, i, i + 1);

        if (odd.size() > longest.size())
            longest = odd;

        if (even.size() > longest.size())
            longest = even;
    }

    return longest;
}

int main() {
    string s = "babad";

    cout << longestPalindrome(s);

    return 0;
}

```

## 19. Container With Most Water

You are given an array where each element represents the height of a vertical line. Find two lines that together with the x-axis form a container holding the maximum water.

```

#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

```

```
int maxArea(vector<int>& height) {  
    int left = 0;  
    int right = height.size() - 1;  
    int maxWater = 0;  
  
    while (left < right) {  
        // Calculate width and height  
        int width = right - left;  
        int h = min(height[left], height[right]);  
  
        // Calculate area  
        int area = width * h;  
  
        // Update maximum area  
        maxWater = max(maxWater, area);  
  
        // Move the shorter line  
        if (height[left] < height[right]) {  
            left++;  
        } else {  
            right--;  
        }  
    }  
  
    return maxWater;  
}
```

```

int main() {

    vector<int> height = {1,8,6,2,5,4,8,3,7};

    cout << maxArea(height);

    return 0;

}

```

## 20. 3Sum

Given an integer array `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` such that their sum equals zero.

```

#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

vector<vector<int>> threeSum(vector<int>& nums) {

    vector<vector<int>> result;

    sort(nums.begin(), nums.end());

    for (int i = 0; i < nums.size(); i++) {

        // Skip duplicates for i
        if (i > 0 && nums[i] == nums[i - 1])
            continue;

        int left = i + 1;

```



```

int right = nums.size() - 1;

while (left < right) {

    int sum = nums[i] + nums[left] + nums[right];

    if (sum == 0) {

        result.push_back({nums[i], nums[left], nums[right]});

        left++;

        right--;

        // Skip duplicates for left and right
        while (left < right && nums[left] == nums[left - 1]) left++;
        while (left < right && nums[right] == nums[right + 1]) right--;
    }

    else if (sum < 0) {

        left++;

    }

    else {

        right--;

    }

}

return result;
}

```

```
int main() {  
  
    vector<int> nums = {-1, 0, 1, 2, -1, -4};  
  
    vector<vector<int>> ans = threeSum(nums);  
  
    for (auto &triplet : ans) {  
        cout << "[";  
        for (int i = 0; i < triplet.size(); i++) {  
            cout << triplet[i];  
            if (i < triplet.size() - 1) cout << ",";  
        }  
        cout << "]" << " ";  
    }  
  
    return 0;  
}
```